P1273R0
86 The Absurd (From Exceptions)

Published Proposal, 2018-10-07

This version:
https://wg21.link/p1273r0

Author:
Isabella Muerte

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:
P1273R0

Current Source:
slurps-mad-rips/papers/proposals/86-the-absurd.bs

Abstract

It’s time we take a stand and throw out the ability to throw pointer to members and floating point values.

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Scope and Impact

4 FAQ

4.1 What types are not permitted currently?

4.2 Can’t I place these types that are being removed into a struct?

4.3 Do we really need this?

5 Wording

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial Release 🎉

§ 2. Motivation

To date, the ability to throw nearly all types within C++ has been permitted. However, there is no true purpose in allowing this. Even today, as a community, it is not only discouraged to throw types such as floats or pointer to members, but it is seen as a red flag that *something isn’t right*.

§ 3. Scope and Impact

Removing this ability will reduce the types that must be implemented by vendor ABIs (such as in the case of `std::exception_ptr`). While the C++ standard does not have a concept of an ABI, in reality and practice this is an issue that has typically prevented changes to existing runtime behavior at the behest of vendors, or in the cases where breaking changes were needed, additional work might have been needed by users. Currently, to implement `std::exception_ptr`, one has to implement type erasure for all possible exception types stored. When statically linking to a standard library (whether recommended by a vendor or not), these types are still pulled in even if they are not used.

Effectively, this paper argues that we should permit only the following types to be thrown:

- `IntegralType`s
- function pointers (such as `void (*)()`)

- User defined types

This list of types is kept for existing practices of error handling in addition to user defined types. `IntegralType`s are permitted because SEH on Windows can in some cases bubble up into an `int`. Additionally, some error messages are thrown as string literals and "error handlers" are in the form of function pointers. While the author disagrees with the approach for all of the above, the fact remains that they are currently in use. However, there is no use or purpose in permitting throwing a float, double, or pointer to member (both data and functions).

§ 4. FAQ

§ 4.1. What types are not permitted currently?

At present, it is considered ill-formed to throw an abstract class type, incomplete type, or any pointer that is not cv void. Some compilers permit additional types such as string literals or function pointers.

§ 4.2. Can’t I place these types that are being removed into a struct?

Go hog wild. Compilers will only generate the data needed for those types when actually needed, rather than having them embedded in an ABI runtime.

§ 4.3. Do we really need this?

Yes. We have compile time type constraints in the form of Concepts, we have runtime requirements in the form of Contracts. We do not currently have a way to prevent someone from ignoring both of these to throw something they should not. Effectively, I can limit the inputs and outputs of a function or a callback passed into a function, but I cannot limit the escape hatch of an exception that can ignore those types. This is a hole in our interfaces and we should patch it up as much as we can.

> Note: While a callback can have `noexcept` attached to it now, this means that a user is unable to pass in a callback that could throw to one of several exception types and if an implementation for a constrained interface does not have a `catch (...)`, this provides potential unexpected behavior that violates what the implementor desired. Also, it’s just gross. Why can anyone throw a `NaN`? It boggles the author’s mind.

§ 5. Wording

The following wording is to be placed (according to [N4762]) in Section 13.1.3

> [3]Throwing an exception copy-initializes (9.3, 10.3.5) a temporary object, called the exception object. An lvalue denoting the temporary is used to initialize the variable declared in the matching handler (13.3). If the type of the exception object would be an incomplete type, an abstract class type (10.6.3), a floating point object, a pointer to member object, or a pointer to an incomplete type other than cv void the program is ill-formed

§ References

§ Informative References

[N4762]
Richard Smith. Working Draft, Standard for Programming Language C+. 7 July 2018. URL: https://wg21.link/n4762